Final Project: Automated Self Balancing Platform; Thrust Vector Control

Abstract

The project's objective is to design and build an autonomous self-balancing platform that, like a thrust vector control system in a missile, holds stability in the face of disturbances. The system uses an MPU6050 IMU (Inertial Measurement Unit) sensor to track motion and two servo motors to readjust the platform's tilt. The PID controller is used to make precise and seamless adjustments. The project demonstrates how automation may combine mechanical motion, control methods, and sensor data to address real-world problems.

Introduction

The self-balancing platform is a useful tool in control systems and robotics. A few examples include stabilizing camera mounts, robotic arms, and autonomous vehicles. This project constructs a small-scale model to illustrate how self-balancing systems function. The tilt data from the MPU6050 sensor is processed in real-time, allowing the platform to be leveled and modified.The platform can spin freely because its frame was 3D printed with pivot points. The objective is to demonstrate how mechanical movements and control theory may be used to address real-world problems. This project demonstrates how combining motion tracking data with careful design and testing will result in controls that are accurate and responsive. The major objective of this project is to create a workable self-balancing platform that remains level despite external effects. For this, the system uses angle data from the MPU6050 sensor, which combines an accelerometer and gyroscope. Combining this data with a complementary filter yields accurate results. The PID controller uses servo motors to tilt the platform in response to these readings. These elements work together to keep the platform stable and shock-responsive.

Hardware and Components

Microcontroller: Arduino Uno

IMU Sensor: MPU6050 accelerometer and gyroscope

Actuators: Two servo motors (SG90)

Platform: A 3D-printed surface with pivot points for free rotation.

Power Supply: 5V DC power supply for the servos and microcontroller.

Miscellaneous: Wires, connectors, and mounting brackets.

Methodology

The MPU6050 sensor needed to be calibrated first. Calibration ensures the accuracy of the accelerometer and gyroscope measurements by accounting for small errors. This was achieved by taking several readings while the sensor stayed stationary and taking offsets into consideration.After calibration, the platform's tilt angles were calculated. A complementary filter was used to integrate the gyroscope's short-term accuracy and the accelerometer's long-term stability. The filter generates precise angle readings that the system may use by balancing these two data streams.A PID controller was used to regulate the platform's orientation. The difference between the desired and actual angles is known as the angle error, and the controller adjusts the servo motors accordingly.

The proportional term reacts to current errors, the integral term corrects accumulated errors, and the derivative term prevents future errors by predicting trends. Tuning the PID gains required multiple tests to find the right balance between stability and responsiveness.The servo motors then acted on the PID output. These motors adjusted the platform’s tilt by rotating in response to the calculated corrections. By mapping the PID outputs to the servo’s range of motion, the system was able to respond quickly to disturbances, maintaining a horizontal position. The integral term fixes cumulative errors, the proportional term reacts to current errors, and the derivative term predicts trends to prevent future errors. To determine the optimal ratio of stability to responsiveness, a number of experiments were carried out to modify the PID gains.The servo motors then had an impact on the PID output. These motors turned as a result of the estimated alterations, altering the platform's tilt. The system mapped the PID outputs to the servo's range of motion, maintaining a horizontal position while responding quickly to perturbations.

Additional Experimentation

Using a NEMA 17 stepper motor and an A4988 stepper motor driver, a propeller at the platform's end was utilized to attempt to simulate a missile's thrust. The thrust was supposed to be added as an additional disturbance to the system in order to give a more realistic representation of thrust vector control. Unfortunately, the Amazon components arrived late, leaving little time for proper integration and testing. The Arduino Uno didn't have enough ports to accommodate the stepper motor driver in addition to the other components, which made implementation even more challenging. Future versions could circumvent these issues by adding a second CPU for additional functionality or a microcontroller with more I/O ports.

Results

Despite multiple interruptions, the self-balancing platform remained stable. It showed minimal oscillations when tilted, recovered quickly, and quickly returned to a horizontal position. The angle error stayed within ±2 degrees, indicating the method's precision. The platform also reacted rapidly, implementing changes in real time with no delay. These results showed how well the complementary filter and PID controller worked together to stabilize the platform.

Challenges

Many problems surfaced along the procedure. Sensor noise was one of the primary issues. It was necessary to thoroughly calibrate the MPU6050 sensor and set the associated filter in order to ensure accurate and unambiguous results. Another issue was servo jitter, which was caused by fluctuations in power and unpredictable signals. By adjusting the PID and providing a dependable power source, this problem was mitigated. Finally, to achieve an appropriate balance between stability and responsiveness, the PID controller needs to be adjusted across a considerable number of testing cycles.

Applications

There are numerous uses for this self-balancing platform. Robotic platforms can move smoothly over uneven terrain because of their stabilizing capabilities. It can maintain stability in a camera mount, resulting in clear photos and/or smooth videos. Since it demonstrates how sensors, controllers, and actuators collaborate in a real-world system, the system is also an excellent tool for education.

Conclusion

In this project, a self-balancing platform was successfully constructed using servo motors, a PID controller, and an MPU6050 sensor. When there were disruptions, the technology was able to swiftly respond to maintain the system level and precisely track motion. It offers a solid foundation for developing increasingly complex stabilizing systems and demonstrates how control systems may be used to real-world issues.

There are several ways to make this project better. A Kalman filter could further reduce sensor noise and increase system accuracy. Expanding the concept to govern three degrees of freedom might allow for the stabilization of increasingly complicated systems. To increase the precision and smoothness of the movements, stepper motors could be utilized in place of the servo motors. Finally, adding Bluetooth or Wi-Fi wireless controllers could provide remote system monitoring and operation.

References

MPU6050 Datasheet and Technical Manual.

Arduino PID Library Documentation.

Control Systems Engineering, Norman S. Nise.

Code:

```
#include <Wire.h>
#include <Servo.h>


const int MPU = 0x68; // MPU6050 I2C address
float AccX, AccY, AccZ;
float GyroX, GyroY, GyroZ;
float accAngleX, accAngleY, gyroAngleX, gyroAngleY, gyroAngleZ;
float roll, pitch, yaw;
float AccErrorX, AccErrorY, GyroErrorX, GyroErrorY, GyroErrorZ;
float elapsedTime, currentTime, previousTime;
int c = 0;


// Servo objects
Servo servoX, servoY;


// Servo pins (replace with your actual pin numbers)
const int servoXPin = 9;
const int servoYPin = 10;


// PID controller variables (adjust these as needed)
float Kp = 0.5; // Proportional gain
float Ki = 0.01; // Integral gain
float Kd = 0.1; // Derivative gain


// Target angles (adjust these to your desired starting position)
float targetRoll = 0.0;
float targetPitch = 0.0;


// Error and integral terms
float errorX, errorY, integralX, integralY;
float previousErrorX, previousErrorY;
```

```
// Servo limits (adjust these as needed)
const int minServoPos = 0;
const int maxServoPos = 180;
const int maxControl = 180; // Adjust this based on your PID tuning


// Low-pass filter constants for smoothing gyroscope data
float gyroX_filtered = 0;
float gyroY_filtered = 0;
float gyroZ_filtered = 0;
float alpha = 0.9; // Smoothing factor (close to 1 keeps the previous value, 0 gives
more weight to the current)


// Gyroscope bias (calculated from stationary measurements)
float gyroBiasX = 0;
float gyroBiasY = 0;
float gyroBiasZ = 0;


void setup() {
Serial.begin(19200);
Wire.begin();


Wire.beginTransmission(MPU);
Wire.write(0x6B);
Wire.write(0x00);
Wire.endTransmission(true);


calculate_IMU_error();
delay(20);


// Initialize servos
servoX.attach(servoXPin);
servoY.attach(servoYPin);


// Set initial servo positions
servoX.write(90); // Adjust this to your desired starting position
```

```
servoY.write(90);


// Calculate and apply gyro bias
calculateGyroBias();
}


void loop() {
// Read accelerometer data
Wire.beginTransmission(MPU);
Wire.write(0x3B);
Wire.endTransmission(false);
Wire.requestFrom(MPU, 6, true);


AccX = (Wire.read() << 8 | Wire.read()) / 16384.0;
AccY = (Wire.read() << 8 | Wire.read()) / 16384.0;
AccZ = (Wire.read() << 8 | Wire.read()) / 16384.0;


// Calculate Roll and Pitch from accelerometer data
accAngleX = (atan(AccY / sqrt(pow(AccX, 2) + pow(AccZ, 2))) * 180 / PI) - AccErrorX;
accAngleY = (atan(-1 * AccX / sqrt(pow(AccY, 2) + pow(AccZ, 2))) * 180 / PI) +
AccErrorY;


// Read gyroscope data
previousTime = currentTime;
currentTime = millis();
elapsedTime = (currentTime - previousTime) / 1000;


Wire.beginTransmission(MPU);
Wire.write(0x43);
Wire.endTransmission(false);
Wire.requestFrom(MPU, 6, true);


GyroX = (Wire.read() << 8 | Wire.read()) / 131.0 + GyroErrorX;
GyroY = (Wire.read() << 8 | Wire.read()) / 131.0 + GyroErrorY;
GyroZ = (Wire.read() << 8 | Wire.read()) / 131.0 + GyroErrorZ;
```

```
// Apply low-pass filter to gyro data to smooth it
gyroX_filtered = alpha * gyroX_filtered + (1 - alpha) * (GyroX - gyroBiasX);
gyroY_filtered = alpha * gyroY_filtered + (1 - alpha) * (GyroY - gyroBiasY);
gyroZ_filtered = alpha * gyroZ_filtered + (1 - alpha) * (GyroZ - gyroBiasZ);


// Update angles with filtered gyroscope data
gyroAngleX = gyroAngleX + gyroX_filtered * elapsedTime;
gyroAngleY = gyroAngleY + gyroY_filtered * elapsedTime;
yaw = yaw + gyroZ_filtered * elapsedTime;


// Complementary filter - combine accelerometer and gyro angle values
roll = 0.96 * gyroAngleX + 0.04 * accAngleX;
pitch = 0.96 * gyroAngleY + 0.04 * accAngleY;


// Calculate error and update integral terms
errorX = targetRoll - roll;
errorY = targetPitch - pitch;
integralX += errorX * elapsedTime;
integralY += errorY * elapsedTime;


// Prevent integral windup
float maxIntegral = 50.0;
integralX = constrain(integralX, -maxIntegral, maxIntegral);
integralY = constrain(integralY, -maxIntegral, maxIntegral);


// PID calculations
float controlX = - (Kp * errorX + Ki * integralX + Kd * (errorX - previousErrorX) /
elapsedTime);
float controlY = Kp * errorY + Ki * integralY + Kd * (errorY - previousErrorY) /
elapsedTime;


// Map control output to servo positions
int servoXPos = constrain(map(controlX, -180, 180, 0, 180), 0, 180);
int servoYPos = constrain(map(controlY, -180, 180, 0, 180), 0, 180);
```

```
// Set servo positions
servoX.write(servoXPos);
servoY.write(servoYPos);


// Reset integral if close to target
if (abs(errorX) < 1.0 && abs(errorY) < 1.0) {
  integralX = 0;
  integralY = 0;
}


// Update previous error for derivative term
previousErrorX = errorX;
previousErrorY = errorY;
}


// Function to calculate gyroscope bias by averaging stationary readings
void calculateGyroBias() {
long sumGyroX = 0, sumGyroY = 0, sumGyroZ = 0;


for (int i = 0; i < 100; i++) {
  Wire.beginTransmission(MPU);
  Wire.write(0x43);
  Wire.endTransmission(false);
  Wire.requestFrom(MPU, 6, true);


  GyroX = (Wire.read() << 8 | Wire.read()) / 131.0;
  GyroY = (Wire.read() << 8 | Wire.read()) / 131.0;
  GyroZ = (Wire.read() << 8 | Wire.read()) / 131.0;


  sumGyroX += GyroX;
  sumGyroY += GyroY;
  sumGyroZ += GyroZ;
  delay(10); // Small delay for smoother readings
}
```

```
gyroBiasX = sumGyroX / 100;
gyroBiasY = sumGyroY / 100;
gyroBiasZ = sumGyroZ / 100;
}


void calculate_IMU_error() {
// Initialize error values
AccErrorX = 0.0;
AccErrorY = 0.0;
GyroErrorX = 0.0;
GyroErrorY = 0.0;
GyroErrorZ = 0.0;


// Code to calculate IMU error
// ...
}
```

CAD & Wiring:

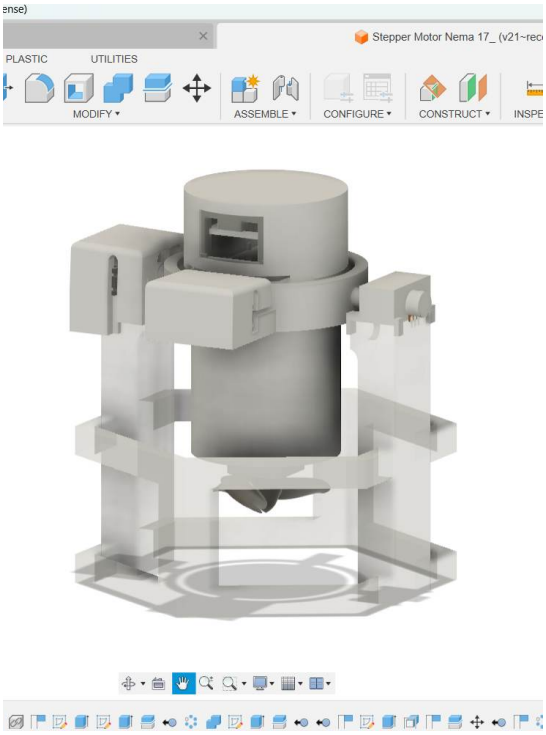


Figure 1 (Isometric View)

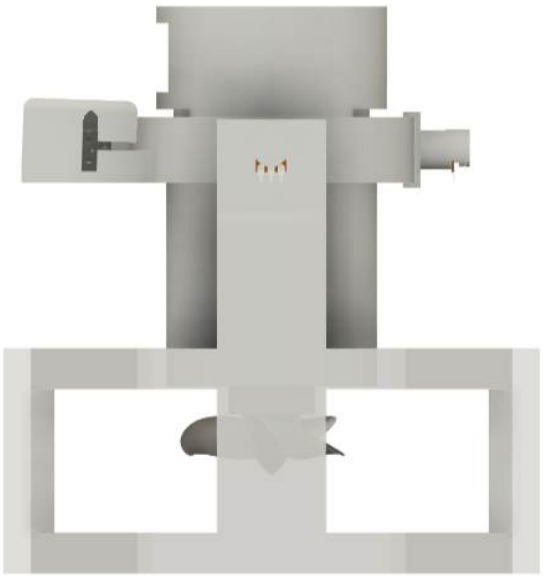


Figure 2 (Side View)

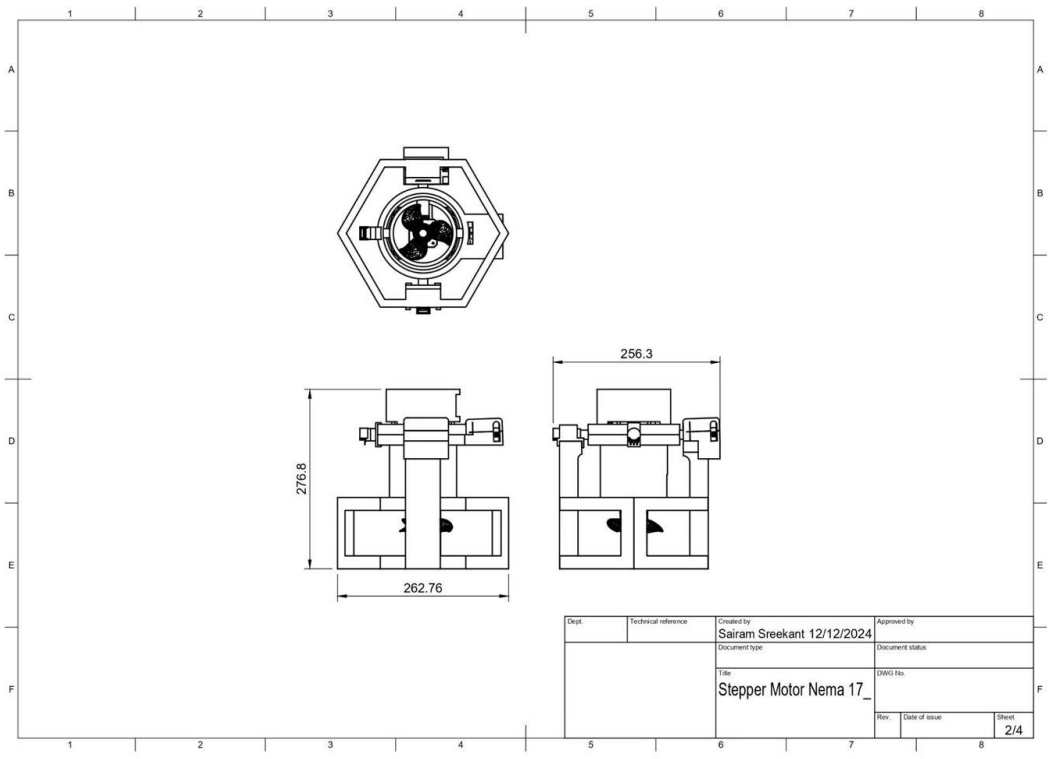


Figure 3 (Assembly)

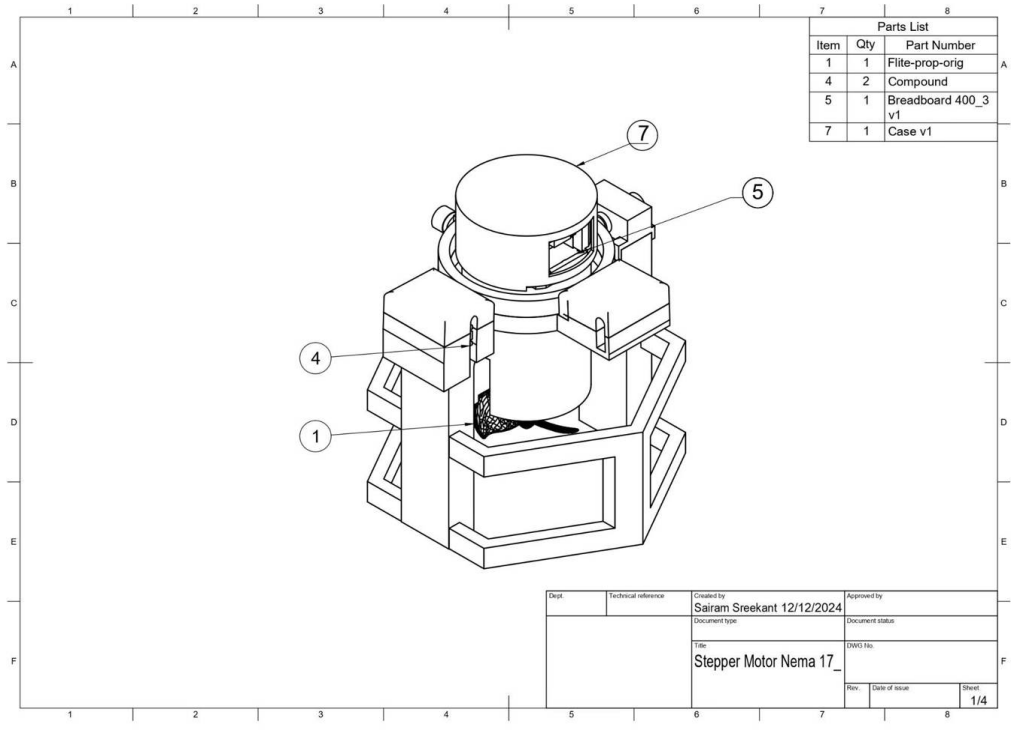


Figure 4 (Isometric Assembly)

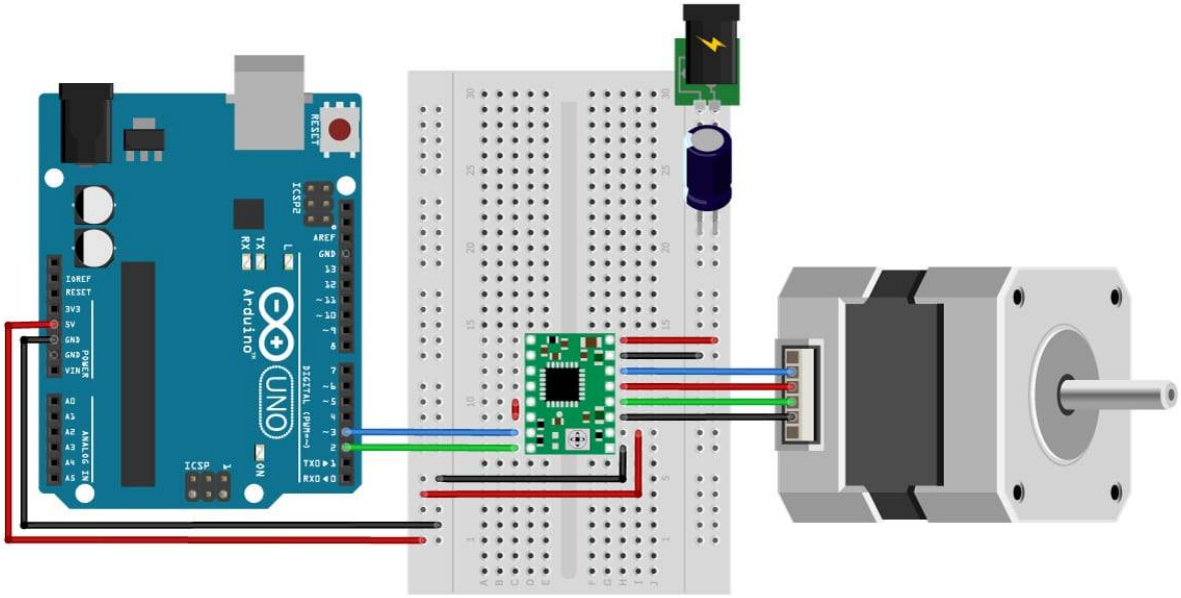
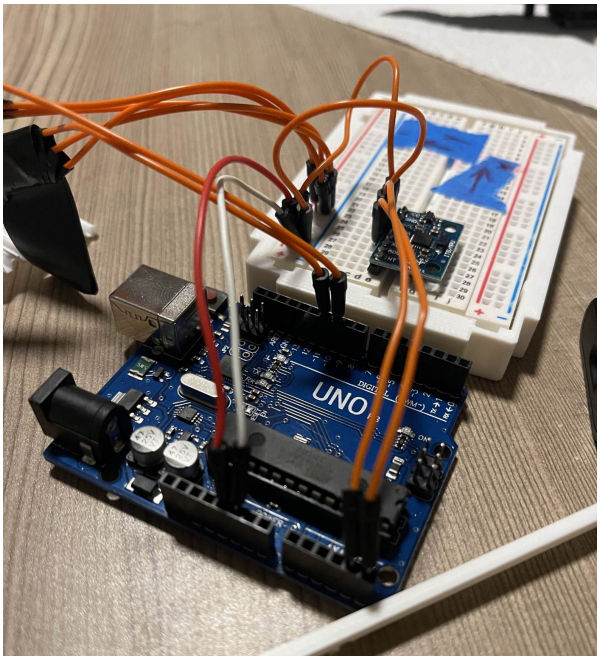
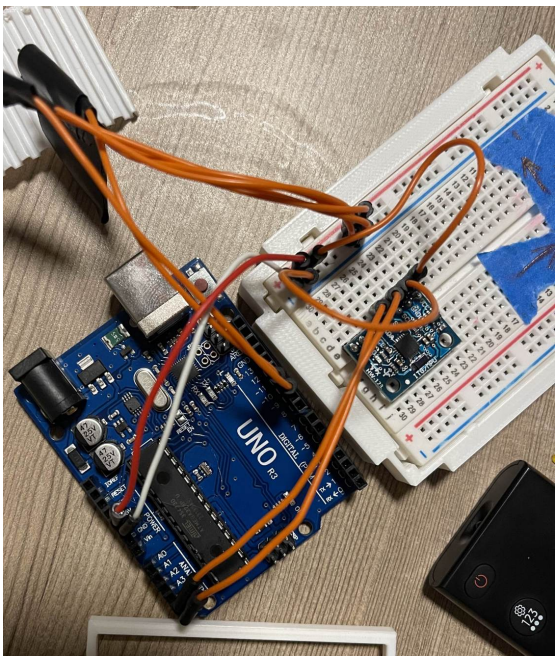
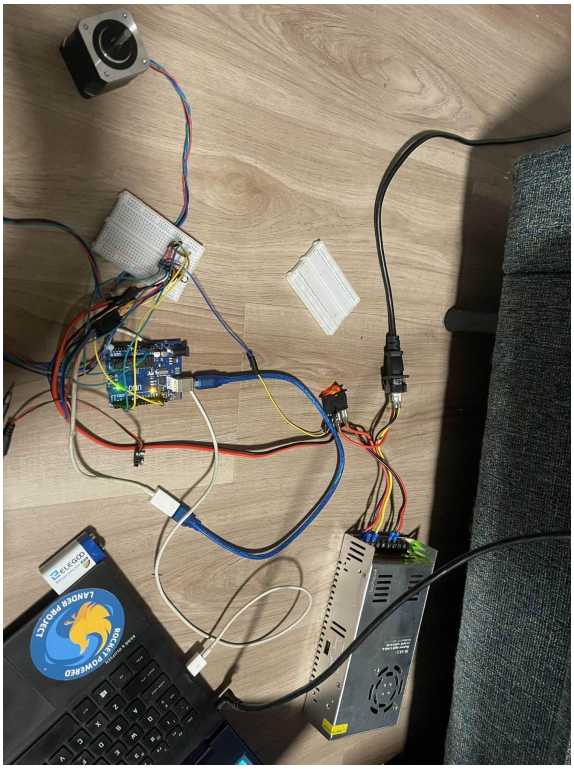
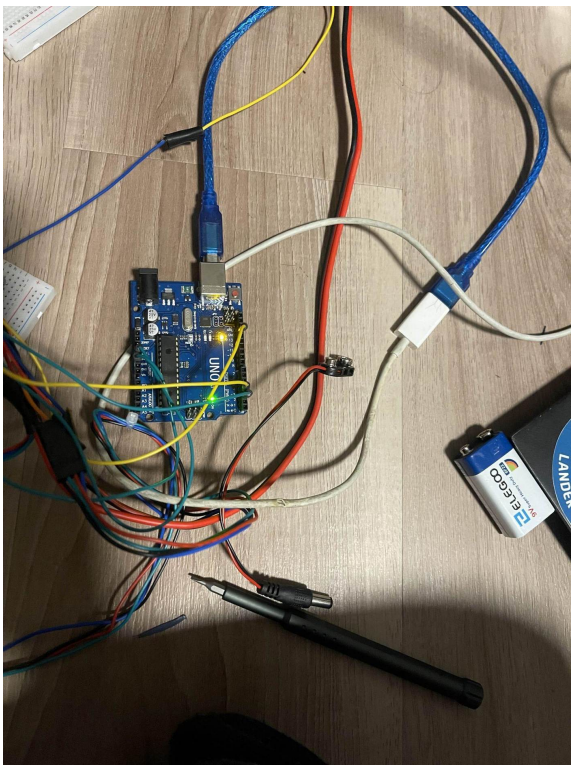


Figure 5 (Partial Wiring Top-Down View)


UNO


UNO




UNO

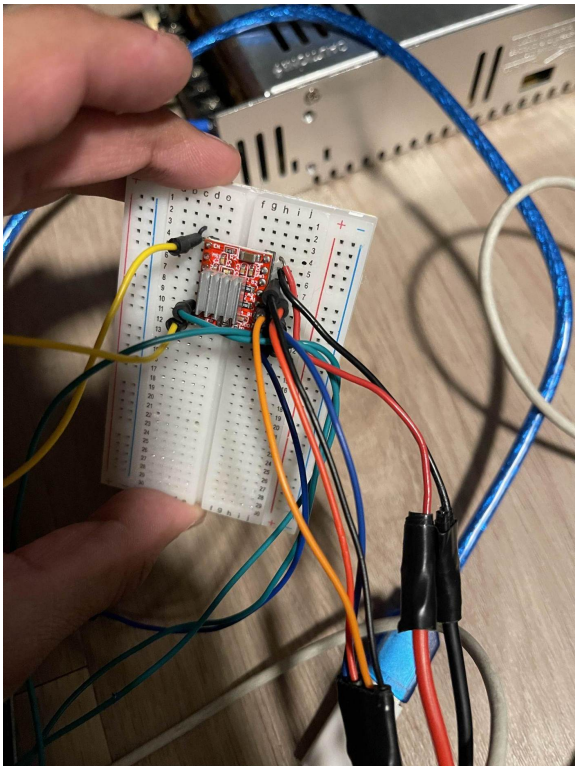


Figure 6 (Collection of 5 Photos, for Reference)